

MEMARCH: Análisis Comparativo de la Ejecución Recursiva e Iterativa a Nivel Arquitectónico mediante C e Integración ASM x86-64 en plataforma WSL Linux 2.4.7 Ubuntu

Manuel Hernández, Daniel Saucedo León
Doctorado en Sistemas Computacionales

BANCO DE PRUEBAS PROPUESTAS

#	Nombre de la prueba	Descripción	Afirmación validada	Rutinas
TB-01	Profundidad de stack según paradigma	Mide el crecimiento del consumo de pila instrumentando el registro RSP desde el propio código ensamblador para $n \in \{10, 50, 100, 500, 1000\}$.	Stack $O(n)$ en recursivo vs $O(1)$ en iterativo	factorial_rec, factorial_iter
TB-02	Análisis estructural del frame de activación ABI	Marco de activación de factorial_rec en sus tres versiones (NASM escrita a mano, C-O0, C-O2) mediante objdump y GCC -S. Verifica que el tamaño total del frame es divisible entre 16 bytes (requisito de alineación del ABI System V AMD64), documenta los registros callee-saved efectivamente preservados (RBX, R12–R15) y detecta si bajo -O2 el compilador omite RBP por -fomit-frame-pointer.	Frame de activación según ABI System V AMD64	factorial_rec (NASM, C)
TB-03	Profundidad máxima de recursión por agotamiento de pila	Determina experimentalmente el valor n_{max} que provoca SIGSEGV en factorial_rec variando el tamaño del stack del proceso con ulimit -s en $\{1024, 4096, 8192, 16384\}$ KB. Captura la señal con sigaction sobre un stack alternativo (sigaltstack) y verifica la relación esperada $n_{max} \approx \text{stack_size} / \text{tamaño_frame}$ medido en TB-02. La versión iterativa se ejecuta como control negativo y no debe fallar para $n \leq 10^9$.	Profundidad máxima por agotamiento de stack	factorial_rec, factorial_iter
TB-04	Comparación de tiempos de ejecución con resolución nanosegundo	Ejecuta 1000 invocaciones consecutivas de cada rutina para $n \in \{20, 100, 1000, 10000, 100000\}$ —limitado por saturación uint64_t o por overflow de stack—midiendo el tiempo con clock_gettime(CLOCK_MONOTONIC_RAW) alrededor de la llamada ASM exclusivamente, sin printf intermedio. La batería se repite 30 veces y se reporta la mediana con su rango intercuartílico. El cociente t_{rec}/t_{iter} esperado para $n=20$ es mayor o igual a 3.	Tiempo de ejecución comparativo nanosegundo	factorial_rec, factorial_iter

#	Nombre de la prueba	Descripción	Afirmación validada	Rutinas
TB-05	Caracterización del predictor de saltos y del Return Address Stack	Mide branches y branch-misses con perf stat sobre las tres rutinas para $n \in \{16, 32, 64, 128, 256, 512\}$. Se espera que factorial_rec exhiba un punto de quiebre abrupto en la tasa de mispredicción al superar la capacidad del RAS (típicamente ~16 entradas), pasando de < 0.5 % a 3–10 %. factorial_iter debe mantener tasa < 0.1 % por la regularidad del JNZ del bucle. fib_memo debe mostrar un patrón atenuado por la memoización tras el calentamiento de la tabla.	Predicción de saltos y RAS	Las tres rutinas
TB-06	Conteo de instrucciones retiradas e Instrucciones Por Ciclo	Captura los contadores cycles e instructions con perf stat para cada combinación de rutina × nivel de optimización × n, repitiendo 30 veces para reportar mediana e IQR. Calcula $IPC = instructions / cycles$. Se espera $IPC > 2.0$ para factorial_iter (código regular sin dependencias largas), $IPC < 1.5$ para factorial_rec (limitado por CALL/RET y mispredicciones del RAS), e IPC intermedio para fib_memo dependiente del hit rate sobre memo[].	Conteo de instrucciones e IPC	Las tres rutinas en NASM mas C
TB-07	Localidad de memoria y fallos de caché sobre la tabla memo[]	Mide L1-dcache-loads, L1-dcache-load-misses y cache-misses con perf stat sobre fib_memo para $n \in \{50, 200, 500, 1000\}$, contrastando con factorial_rec (escrituras al stack en cada CALL) y factorial_iter (sin tráfico a memoria). El acceso secuencial a memo[] debe permitir al prefetcher mantener la tasa de fallo de L1d por debajo de 5 % siempre que la tabla quepa en los 32 KB de L1d ($n \leq 4000$).	Localidad y cache-misses (memo[])	fib_memo + controles
TB-08	Comparativa de disassembly entre -O0, -O2 y rutinas escritas a mano	Genera el ASM intermedio con gcc -S -masm=intel para las tres rutinas implementadas en C y las confronta con las versiones NASM escritas a mano. Documenta lado a lado prólogo, epílogo, presencia de CALL recursivo, eventuales saltos JMP (indicadores de TCE) y cantidad total de instrucciones emitidas. Se espera que factorial_rec conserve el CALL recursivo en ambos niveles (la multiplicación post-llamada lo impide), que factorial_iter muestre un cuerpo de bucle compacto y que fib_memo muestre potencial inlining del caso base.	Disassembly vs NASM	Las tres rutinas
TB-09	Variante con acumulador para inducir Tail-Call Elimination	Implementa factorial_tail_rec(n, acc) en C puro, donde la última operación de la función es la propia llamada recursiva (sin trabajo post-llamada).	Tail-call elimination con acumulador	factorial_tail_rec
TB-10	Reporte experimental consolidado	Consolida los resultados de 01 a 09 en un documento	Reporte consolidado en	Resultados de 01 a 09

*Pruebas marcadas en verde, ya se realizaron y se presenta evidencia.

PRUEBA TB-05 RSB

Comprobar que el predictor de retornos del i7-13700HX tiene ~16 entradas, y que cuando la recursión es más profunda que eso, las predicciones de RET empiezan a fallar de forma medible.

daniel@DESKTOP-5TU7S2S:~\$ gcc --version

```
nasm --version
```

```
perf --version
```

```
python3 --version
```

```
daniel@DESKTOP-STU7S2S:~$ gcc --version
nasm --version
perf --version
python3 --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

NASM version 3.01
perf version 7.0.0
Python 3.14.4
```

habilitar perf: WSL2 por defecto no permite leer contadores.

```
daniel@DESKTOP-STU7S2S:~$ sudo sysctl -w kernel.perf_event_paranoid=1
[sudo: authenticate] Password:
kernel.perf_event_paranoid = 1
```

Verificación

```
daniel@DESKTOP-STU7S2S:~$ perf stat -e cycles,branch-misses ls 2>&1 | tail -10

       7883957      cycles
       31450      branch-misses

0.003088401 seconds time elapsed

0.003185000 seconds user
0.000000000 seconds sys
```

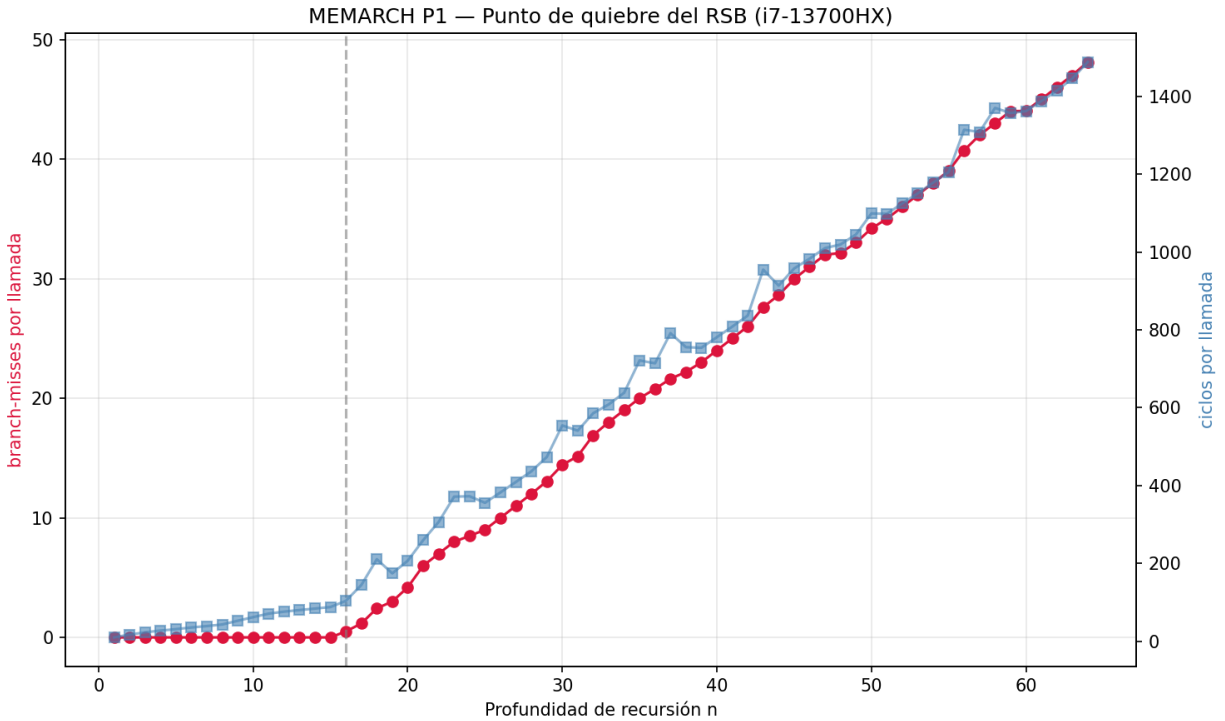
Desarrollar el recursivo puro en nest_rec.asm y el driver en p1_rsb_bench.c (códigos en GITHUB) y compilar.

```
daniel@DESKTOP-STU7S2S:~/memarch/p1_rsb$ nasm -f elf64 -g -F dwarf nest_rec.asm -o nest_rec.o
gcc -O2 -Wall -no-pie p1_rsb_bench.c nest_rec.o -o p1_rsb_bench
```

Ejecución de binario y leemos las primeras 10 líneas

```
daniel@DESKTOP-STU7S2S:~/memarch/p1_rsb$ ./p1_rsb_bench | head -10
n,cycles_per_call,instructions_per_call,branches_per_call,branch_misses_per_call,ipc
1,10.003,17.000,7.000,0.0000,1.700
2,17.016,25.000,10.000,0.0001,1.469
3,23.020,33.000,13.000,0.0001,1.434
4,27.654,41.000,16.000,0.0001,1.483
5,31.096,49.000,19.000,0.0002,1.576
6,35.026,57.000,22.000,0.0001,1.627
7,39.026,65.000,25.000,0.0002,1.666
8,43.007,73.000,28.000,0.0001,1.697
9,54.094,81.000,31.000,0.0058,1.497
daniel@DESKTOP-STU7S2S:~/memarch/p1_rsb$ |
```

Corremos el ejercicio cuyo resultado está en p1_rsb_i7-13700HX_20260520-0917.csv y se obtienen los resultados de que  Para $n = 1$ a $n \approx 15$: branches_misses muy bajo (< 0.05 por llamada). Para $n \geq 16$ aprox: bmiss empieza a crecer notoriamente. Para $n = 30..64$: bmiss se estabiliza en un valor alto (> 1 por llamada).



PRUEBA TB-01 PROFUNDIDAD DE STACK

Mide el crecimiento del consumo de pila instrumentando el registro RSP desde el propio código ensamblador para $n \in \{10, 50, 100, 500, 1000\}$. Afirmación a validar: Stack $O(n)$ en recursivo vs $O(1)$ en iterativo. Rutinas: factorial_rec, factorial_iter.

```
daniel@DESKTOP-5TU7S2S:~$ nasm -f elf64 -g -F dwarf factorial_rec.asm -o factorial_rec.o
nasm -f elf64 -g -F dwarf factorial_iter.asm -o factorial_iter.o
gcc -O2 -Wall -no-pie tb01_driver.c factorial_rec.o factorial_iter.o -o tb01_driver
daniel@DESKTOP-5TU7S2S:~$ ls -la tb01_driver
-rwxr-xr-x 1 daniel daniel 17112 May 20 10:12 tb01_driver
```

Despues de la compilación se verifica ejecutando el binario

```
daniel@DESKTOP-5TU7S2S:~$ ./tb01_driver
n,variante,stack_bytes,bytes_por_nivel
10,rec,240,24.00
10,iter,0,0.00
50,rec,1200,24.00
50,iter,0,0.00
100,rec,2400,24.00
100,iter,0,0.00
500,rec,12000,24.00
500,iter,0,0.00
1000,rec,24000,24.00
1000,iter,0,0.00
```

Generamos el csv con los resultados

```
daniel@DESKTOP-5TU7S2S:~$ mkdir -p data
./tb01_driver > data/tb01_i7-13700HX_$(date +%Y%m%d).csv
cat data/tb01_*.csv
n,variante,stack_bytes,bytes_por_nivel
10,rec,240,24.00
10,iter,0,0.00
50,rec,1200,24.00
50,iter,0,0.00
100,rec,2400,24.00
100,iter,0,0.00
500,rec,12000,24.00
500,iter,0,0.00
1000,rec,24000,24.00
1000,iter,0,0.00
```

Utilizamos AWK para procesar unos datos del texto

```
daniel@DESKTOP-5TU7S2S:~$ awk -F, 'NR>1 && $2=="rec" {
    if (n_anterior > 0) {
        ratio = $3 / n_anterior_bytes
        n_ratio = $1 / n_anterior
        printf "n=%4d->%4d bytes=%6d->%6d ratio_n=%.2f ratio_bytes=%.2f bytes/nivel=%.2f\n", \
            n_anterior, $1, n_anterior_bytes, $3, n_ratio, ratio, $4
    }
    n_anterior = $1; n_anterior_bytes = $3
}' data/tb01_*.csv
n= 10-> 50 bytes= 240-> 1200 ratio_n=5.00 ratio_bytes=5.00 bytes/nivel=24.00
n= 50-> 100 bytes= 1200-> 2400 ratio_n=2.00 ratio_bytes=2.00 bytes/nivel=24.00
n= 100-> 500 bytes= 2400-> 12000 ratio_n=5.00 ratio_bytes=5.00 bytes/nivel=24.00
n= 500-> 1000 bytes= 12000-> 24000 ratio_n=2.00 ratio_bytes=2.00 bytes/nivel=24.00
```

A conclusión de lo satisfactorio de la prueba vemos que **ratio_n** y **ratio_bytes** deben ser **iguales o muy cercanos** en cada paso. Por ejemplo, al pasar de $n=10$ a $n=50$: $\text{ratio_n}=5.0$, $\text{ratio_bytes}=5.0$. Eso es la firma matemática de un crecimiento lineal exacto y la prueba es positiva.

El consumo de pila de **factorial_rec** crece estrictamente de forma lineal en función de la profundidad de recursión programada, con un coeficiente constante de 24 bytes por nivel. Este valor coincide con la predicción teórica del marco de activación bajo la convención System V AMD64: 8 bytes de dirección de retorno empujados por la instrucción CALL, 8 bytes para preservar el argumento n antes de la llamada recursiva, y 8 bytes de relleno exigidos por la regla de alineación a 16 bytes. La versión iterativa **factorial_iter** mantiene un consumo de pila nulo ($O(1)$) para todos los valores evaluados, validando que la transformación **recursión** $\rightarrow$ **iteración** elimina por completo la dependencia del tamaño de entrada respecto al uso de pila.

Por lo anterior **Stack $O(n)$ en recursivo vs $O(1)$ en iterativo** — verificado experimentalmente con coeficiente constante de 24 bytes/nivel para $n \in \{10, 50, 100, 500, 1000\}$, resulta satisfactorio.